

Projeto de Programação em Python

UFCD/Disciplina: 10790 - Projeto de Programação

Professor: Carlos Barata

Aluno: Bruno Henses

Data: 01/08/2025

A seguir estão os requisitos funcionais (RF) e não funcionais (RNF) do jogo "Show do Milhão Dinâmico", organizados em formato de tabela para melhor clareza.

Requisitos Funcionais (RF)			
ID	Descrição	Prioridade	Observações
RF01	Exibir tela inicial com botão "Iniciar Jogo"	Alta	Desabilitar botão após clique
RF02	Carregar perguntas de dificuldade progressiva (4 fáceis, 3 médias, 3 difíceis)	Alta	Total de 10 perguntas por partida
RF03	Oferecer modo offline: usar apenas JSON local quando OPENAI_API_KEY não estiver definido	Alta	Mostrar indicador de modo offline
RF04	Gerar perguntas dinâmicas via OpenAI API quando chave válida	Média	Usar gpt-4o-mini com temperatura configurável
RF05	Evitar repetição imediata de perguntas em cada partida	Média	Implementar cache local (cache.json)
RF06	Exibir pergunta atual com até 5 opções de múltipla escolha	Alta	Texto ajustável (wraplength)
RF07	Permitir seleção de resposta clicando em botão	Alta	Interface construída com ttkbootstrap
RF08	Validar resposta e atualizar prêmio garantido	Alta	Sequência de patamares crescente
RF09	Avançar de nível após acertos (fácil → média → difícil)	Alta	Jogo termina ao concluir ou errar
RF10	Fornecer feedback imediato (correto/incorreto) na UI	Alta	Exibir valor atual e mensagem
RF11	Finalizar jogo ao errar, exibindo resultado final e opção de reiniciar	Alta	Botão "Reiniciar Jogo"
RF12	Disponibilizar ajuda "Pular Pergunta" uma vez por partida	Baixa	Contador de usos
RF13	(Stretch) Disponibilizar ajuda "50/50" para eliminar duas alternativas incorretas	Baixa	Ativar se tempo permitir
RF14	(Stretch) Disponibilizar ajuda "Dica" que fornece uma pista sobre a resposta correta	Baixa	Sugestão gerada com base na pergunta atual

Resquisitos Não Funcionais (RNF)			
ID	Descrição	Prioridade	Observações
RNF01	Interface gráfica responsiva, baseada em ttkbootstrap	Alta	Compatível com Windows, macOS e Linux
RNF02	Tempo de resposta ao carregar e apresentar pergunta < 1s	Média	Depende de desempenho local ou latência da API
RNF03	Organização modular: main.py, question_manager.py, game_logic.py, ui.py	Alta	Facilita manutenção e testes
RNF04	Gerenciamento de dependências via requirements.txt	Alta	Inclui openai, python-dotenv, ttkbootstrap
RNF05	Variáveis sensíveis (chave API) em .env com .gitignore	Alta	Protege credenciais
RNF06	Estilo de código seguindo PEP8	Média	Usar flake8 para lint
RNF07	Log de eventos e erros em arquivo local	Baixa	Registrar falhas de API e exceções
RNF08	Documentação clara e passo-a-passo para iniciantes (README.md)	Alta	Atualizado com uso de .env e requisitos
RNF09	Testes básicos automatizados (unitários) para question_manager e game_logic	Baixa	Assegurar integridade da lógica